TECHNICAL DOCUMENTATION

WebGpuWater

Interactive GPU water for Unity URP — the complete system reference: architecture, every module in depth, and engineering notes.

Package com.abstractocclusion.webgpuwater · **Version** 1.0.0
Engine Unity 2022.2+ / URP 12+ · Desktop · WebGPU/WebGL · Mobile
Generated 20 July 2026 · A modern URP adaptation of Evan Wallace's WebGL Water (MIT)

Contents

Introduction

AbstractOcclusion.WebGpuWater is a GPU-driven interactive water system for the Unity Universal Render Pipeline (URP). It began as a faithful Unity 6 / URP port of Evan Wallace's classic *WebGL Water* (2011, MIT) and has since grown into a full native adaptation and a large superset of the original: an interactive ripple simulation, two-way buoyancy, layered ambient waves up to a full FFT ocean, surface and particle foam, caustics, volumetric god rays, hybrid reflections, underwater fog, coastlines and surf, exclusion volumes, and self-contained "chunks" of water — all authored from a single wizard window and all designed to run from desktop down to WebGPU in a mobile browser.

The system's defining engineering constraint is that it targets **WebGPU first**. Every design decision — how textures are formatted, how the height field is read back, how foam particles are spawned, how shaders sample floating-point data — is shaped by what the strictest backend (WebGPU / GLES-class mobile) allows. The payoff is a water package of desktop quality that has been verified running at ~30 fps in a browser on entry-level 2026 tablets.

This document is a complete technical reference for the package. It opens with the architecture and the ideas that recur throughout the codebase, then documents each module in depth. A companion **API Reference** covers the public scripting surface symbol by symbol.

> **Lineage & credit.** The GPU height-field simulation, the in-shader ray-traced reflection/refraction, and the projected caustics are Evan Wallace's original ideas — https://madebyevan.com/webgl-water/. This adaptation is released in the same spirit (MIT). Full credit for the foundational design belongs to him.

Scope & Capabilities

The package is organized around one central component, `WaterVolume` — one per body of water — and a set of collaborating modules it composes at runtime. A single scene can host many independent bodies at once.

What the system covers today:

- **Contained bodies** — pools, ponds, and small-to-mid lakes driven by the interactive ripple grid, with an analytic ray-traced pool floor, tiles, walls, and caustics.
- **Large & open water** — a camera-following simulation window keeps mid-size water crisp, and an opt-in **FFT (Tessendorf) ocean** with a world-locked horizon clipmap extends the system to unbounded oceans with whitecaps, swell, and choppiness.
- **Coastlines** — a baked bed-depth / shoreline field drives deep-water colour, a shoaling gradient, wet/dry clipping, and a physically-modelled surf/breaker layer with swash foam.
- **Exclusion volumes** — Crest-style carving that removes water from a boxed region (a hull interior, a flooded room) while keeping the surrounding fog, shadows, and waterline consistent.
- **Water chunks** — the inverse of a carve: a finite, self-contained body of water (box, sphere, or arbitrary mesh) floating in dry space, with its own fog shell and animated fill level.
- **Two-way physics** — objects float, roll, self-right, and drift with the waves (`WaterBuoyancy`) *and* disturb the surface they ride on (`WaterInteractable`, `WaterObstacle`, `WaterSphereInteractor`), plus entry splashes and scripted ripples.
- **Optics** — caustics on floors and submerged objects, god-ray shafts with *real* shadow casting, hybrid sky / planar / screen-space reflections, screen-space or analytic refraction, and Beer–Lambert underwater fog with Jerlov-derived water colours.

A few paths are explicitly marked **experimental / preview** in the code and demo set: Unity Terrain integration (the bed-depth bake approximates a shoreline from a heightmap) and the open-water ocean demos. Treat those as preview quality.

Requirements & Installation

Engine & pipeline

- **Unity 2022.2 or newer** (Unity 6 fully supported; the host project was developed on Unity 6).
- **URP 12+** for the full look. The base runtime assembly compiles *without* URP installed; URP-only code (planar reflection, the renderer features) activates automatically through the `WEBGPUWATER_URP` scripting define — no manual setup.
- A GPU with **compute shaders** and `RGBAFloat` **random-write** textures. On the web, builds require a **WebGPU-capable browser** (Chrome/Edge, Safari 26+, recent Firefox); unsupported devices get a clear message rather than a crash.

URP asset settings — on the active URP asset, enable:

- **Depth Texture** — required for SSR and screen-space refraction.
- **Opaque Texture** — required for real refraction.
- **Transparent Receive Shadows** — required for god-ray shadow shafts. They render in the transparent queue; with this off, the shafts silently vanish.

Install — add the UPM package `com.abstractocclusion.webgpuwater` (Package Manager ▸ *Install from disk / tarball*, or your registry). Then open **Package Manager ▸ AbstractOcclusion.WebGpuWater ▸ Samples** and import **Demo Scenes**. The package itself stays read-only; everything the wizard generates (meshes, materials, sky, quality asset) is written into your own `Assets/` folder.

Architecture at a Glance

The `WaterVolume`-centric model

`WaterVolume` is the single `MonoBehaviour` that represents one body of water. It is a large `partial class` split across many files by concern (identity & lifecycle, the public façade, the serialized settings, height queries, ocean clipmap, chunk, underwater, obstacle-debug). Its transform *is* the volume's frame: position, rotation, and a per-axis `volumeExtent` define an oriented box with the surface at local `y = 0`, the floor at `y = -1`, and the pool spanning `x, z ∈ [-1, 1]`. Non-uniform extent lets a body be deep, rectangular, or rotated without ever touching object scales.

Rather than a monolith, `WaterVolume` **composes collaborators** through a small `IWaterModule` seam. On enable it builds a fixed set of modules — the simulation, the obstacle rasterizer, the caustics pass, the CPU surface sampler, the FFT ocean, and the sim window — initializing only those whose `Enabled` gate passes for the current configuration. Each module owns its GPU resources and is disposed on disable. This keeps a pool body cheap (no ocean, no window) while the same component scales up to a full ocean.

Multiple bodies, one budget

A static registry tracks every live body (`WaterVolume.Bodies`, `Primary`, `BodyContaining(point)`, `Resolve()`). Because each body pushes its per-body state through a `MaterialPropertyBlock` rather than unique material instances, many bodies share one material and one shader variant set. A floating object is lit and driven by whichever body actually contains it (`WaterMembership`). A frame-guarded `WaterSimScheduler` culls bodies by frustum and distance and caps how many simulate at once (a fixed active-sim budget), so a scene full of lakes stays affordable.

Stacked height sources

The visible surface height is composited from several independent sources, coarse to fine:

1. the **interactive ripple** height field (`WaterSimulation` + `WaterSim.compute`),
2. the **wind-wave** sum-of-sines bank (`WaterWaveBank` + `WaterWaves.hlsl`),
3. the **analytic large-wave** swell + choppy Gerstner field (`WaterLargeWaves.hlsl`) — or, when enabled, the **FFT ocean** cascade that replaces it,
4. the **surf / shore** breaker and swash layer near coastlines.

Each source is a pure function of position and time, evaluated identically wherever it is needed.

The CPU⇄GPU parity principle

Buoyancy must sample *exactly* the surface the eye sees, and it must do so without a per-frame GPU read-back on backends where read-back is unreliable. So the analytic wave math is duplicated on both sides: `WaterWaveBank ↔ WaterWaves.hlsl` and `LargeWaveField ↔ WaterLargeWaves.hlsl / WaterSurfWaves.hlsl` are kept **byte-for-byte identical**, down to shared constants. An editor-load guard, `WaterWaveConstantsValidator`, parses the paired constants and turns any drift into a console error — replacing "remember to edit both files" with a hard check. The interactive ripple and FFT paths, whose height *is* GPU-authored, instead expose a CPU

mirror through an async read-back with an analytic fallback, so floating never stops working even where read-back is absent.

WebGPU as the design floor

Several patterns throughout the code exist purely to satisfy WebGPU/GLES: read-write compute targets are single-channel `r32`; float textures are filtered by hand in shaders (WebGPU point-samples `RGBA32Float`); samplers are always bound to a black fallback rather than left null; shadow maps and scene depth are read with explicit point samplers; volume conservation uses an exact group-shared reduction instead of a mip chain that silently point-samples; and shader stages hoist screen-space derivatives out of non-uniform control flow to satisfy WGSL. These are called out per module where they matter.

Quick Start

With the Water Wizard

Window ▸ AbstractOcclusion ▸ WebGpuWater ▸ Water Wizard is the one-window authoring tool. Pick a base kind (analytic pool, surface-only, surface-with-fog, or open-water ocean), set the size, toggle the features you want (reflection mode, foam, god rays, splash, floor collider), optionally drag scene objects in to make them float or interact, and press **Create Water Surface**. The wizard builds the sim volume, the surface renderers, a splash emitter, a quality asset, and a tweakable material saved into *your* project, then wires up the camera. Press **Play** — drag on the surface to ripple it; drop a Rigidbody with `WaterBuoyancy` in and it floats.

From script

```
using AbstractOcclusion.WebGpuWater;

// Resolve a body
WaterVolume water = WaterVolume.Primary;                 // global fallback body
water = WaterVolume.BodyContaining(worldPoint);          // the body a point sits in

// Query the live surface
if (water.TryGetWaterHeight(x, z, out float surfaceY)) { /* ... */ }
water.TryGetSurface(x, z, out float h, out Vector2 flow);
water.TrySampleSubmersion(point, out float depth, out Vector3 up, out Vector3 flow3);

// Disturb the surface
water.AddRipple(x, z, radius: 0.05f, strength: 0.03f);
water.AddSphereInteraction(pos, worldStep, radius, strength);   // directional wake

// Body-resolving statics (find the right body for you)
WaterVolume.TrySampleHeightAt(pos, out float y);
WaterVolume.TrySpawnRippleAt(pos, radius, strength);

// Runtime look / behaviour (properties)
water.Foam = true;
water.WindWaves = true;
water.WaterFog = true;
water.RippleStrength = 0.03f;
water.TimeScale = 1f;
water.EnableCulling = true;
```

Everything else is tuned on the component in the inspector, which is fully tooltipped. Anything not exposed as a property is intentionally internal — tune it on the component, not from script.

Module Reference

The system is a set of modules that `WaterVolume` composes. The map below groups them; each is documented in full in the sections that follow.

Area	Key files	What it owns
Core & lifecycle	`WaterVolume(.Facade/.Query/.Settings…)`, `WaterUniformPublisher`, `WaterMembership`	The body, its registry, the scripting façade, uniform publishing
Simulation	`WaterSimulation`, `WaterSim.compute`, `WaterSimScheduler`, `WaterSimWindow`	The interactive ripple height field
Waves & ocean	`WaterWaveBank`, `LargeWaveField`, `WaterOceanFft`, `OceanFft.compute`, `LargeWaterClipmap`	Wind waves, swell, FFT ocean, horizon
Physics & interaction	`WaterBuoyancy`, `WaterInteractable`, `WaterObstacle`, `WaterSphereInteractor`, `WaterInputRouter`	Two-way object coupling & input
Foam & particles	`WaterFoamParticles`, `WaterFoamProfile`, `WaterSplash(Emitter)`	Surface foam field, GPU spray, splashes
Rendering & optics	`WaterCausticsPass`, `PlanarMirror`, `Rendering/*`, caustic/god-ray/fog shaders, `WaterQuality`	Caustics, god rays, reflections, underwater fog, tiers
Surface shader	`WaterSurface.shader` + `WaterSurface*.hlsl`	The visible water material
Coastline	`WaterShoreDepthField`, `WaterBedBaker`, `WaterExclusionVolume`, `WaterVolume.Chunk`	Bed depth, exclusion carving, water chunks
Authoring & tooling	`Editor/WaterWizardWindow`, `WaterVolumeEditor.*`, cameras, showcase	The wizard, custom inspector, validators, cameras

Core, Lifecycle & Uniform Publishing

Overview. `WaterVolume` is the single central `MonoBehaviour` for one water body — the "thin master" that owns and orchestrates every collaborator. It is a `partial class` split across `WaterVolume.cs` (identity, lifecycle, sim step), `.Facade.cs` (public gameplay API + GPU-consumer surface), `.Query.cs` (`IWaterHeightSampler` seam), `.Settings.cs` (serialized fields, static registry, migrations), plus `.SimWindowPatch.cs`, `.OceanClipmap.cs`, `.Chunk.cs`, `.Underwater.cs`, `.ObstacleDebug.cs`. It is marked `[ExecuteAlways]` and `[DefaultExecutionOrder(-50)]` so it previews in edit mode and updates before cameras move. It declares `MonoBehaviour, ISerializationCallbackReceiver` and implements `IWaterHeightSampler`. Coordinate convention: surface at `y = 0`, pool spans `x, z ∈ [-1, 1]`, floor at `y = -1`. There is **no** `Awake` — lifecycle runs entirely through `OnEnable` / `Update` / `OnDisable`.

Key types.

- `WaterVolume` — central body component; owns modules, publishes uniforms, exposes the gameplay façade.

- `IWaterModule` (internal) — lifecycle seam: `bool Enabled { get; }`, `Initialize(WaterContext)`, `Dispose()`.
- `WaterContext` (internal sealed) — shared per-frame seam handed to modules; currently carries only `WaterVolume Owner`.
- `WaterCollaboratorModules.cs` — six adapters: `SimulationModule`, `ObstacleModule`, `CausticsModule`, `SurfaceSamplerModule`, `OceanFftModule`, `SimWindowModule`.
- `WaterUniformPublisher` (internal sealed) — single source of truth for per-body uniform derivations, written through an `IUniformSink` (`MpbUniformSink` / `GlobalUniformSink`).
- `WaterMembership` (`MonoBehaviour`, `[ExecuteAlways]`, `[RequireComponent(typeof(Renderer))]`) — associates a floating object with its containing body.
- `WaterShaderNames` (internal static) — registry of shader declaration names (`Root = "AbstractOcclusion/WebGpuWater/"`).
- `WaterMeshBuilder` (internal static) — procedural grid/disc/cube mesh builder.
- Enums on `WaterVolume`: `ObjectInteraction { MouseLikeDrops, FootprintDelta }`, `RippleQuality { Low, Medium, High, Ultra }`, `WaterBodyType { Pond, Lake, Ocean }`.

Lifecycle & registry. `OnEnable` subscribes to `RenderPipelineManager.beginCameraRendering` then calls `TryInitialize`, guarded by `_initialized`. In edit mode, missing wiring is retried each `Update`; in play mode `FailMissingWiring` fails loudly. `TryInitialize` checks compute/float-RT capability, resolves scene references, applies the quality tier, decides `_windowed` via `ShouldWindow()`, then `BuildAndInitializeModules()` constructs the `WaterContext` and the six modules in fixed order, initializing only those whose `Enabled` gate passes. It seeds ripples, registers into the static registry, creates the `MaterialPropertyBlock`, and publishes globals. `OnDisable` unsubscribes, hands off `Primary`, removes from `Bodies`, and disposes modules and the lazy trio.

The **static registry** lives in `WaterVolume.Settings.cs`: `public static WaterVolume Primary { get; private set; }` (global fallback), `internal static readonly List<WaterVolume> Bodies`, `public static WaterVolume Resolve()` (Primary else a frame-cached scene search), and `public static WaterVolume BodyContaining(Vector3 worldPoint)` (footprint containment; nearest horizontal centre wins, else `Resolve()`). A `[RuntimeInitializeOnLoadMethod] ResetStaticState` clears these for fast-enter-play-mode. `ISerializationCallbackReceiver.OnAfterDeserialize` runs versioned migrations.

Scripting façade API (`WaterVolume.Facade.cs` / `.Query.cs`):

Signature	Purpose
`void AddRipple(float worldX, float worldZ, float radius, float strength)`	Inject an isotropic height drop
`void AddSphereInteraction(Vector3 worldPos, Vector3 worldStep, float radius, float strength)`	Directional velocity-dipole wake
`static bool TrySphereInteractionAt(Vector3 worldPos, Vector3 worldStep, float radius, float strength)`	Sphere wake on the containing body
`bool TryGetWaterHeight(float worldX, float worldZ, out float height)`	World surface Y
`bool TryGetSurface(float worldX, float worldZ, out float height, out Vector2 flow)`	Height + surface flow
`bool TrySampleSubmersion(Vector3 worldPoint, out float depthWorld, out Vector3 up, out Vector3 worldFlow)`	Buoyancy depth/normal/flow
`bool TrySampleHeight(Vector3 worldPos, out float worldY)` · `bool IsSubmerged(Vector3)` · `void SpawnRipple(Vector3, float, float)`	Vector wrappers
`static bool TrySampleHeightAt / IsSubmergedAt / TrySpawnRippleAt(...)`	Body-resolving statics
`bool TryGetAnalyticWaterline(float worldX, float worldZ, out float height)`	Rest + wind waterline (valid from frame 0)
`bool SampleHeight(Vector3, out WaterSample, float minimumLength = 0, bool excludeInteractiveRipples = false)`	`IWaterHeightSampler` single point
`void SampleHeights(int ownerHash, float minimumLength, IReadOnlyList<Vector3>, WaterSample[], WaterQueryFields = …, bool = false)`	Batched query
`static bool SampleHeightAcrossBodies(Vector3, out WaterSample, float minimumLength = 0)`	Per-point body resolution
`bool TryRaycastSurface(Ray, out Vector3 worldHit)`	Input-router pick
`void WriteBodyProps(MaterialPropertyBlock mpb)` · `void WriteSimFrameUniforms(ComputeShader cs)`	Push uniforms out

Façade **properties** include: `RenderTexture SimStateTexture`, `RenderTexture FoamMaskTexture`, `int SimResolution`, `bool IsSimulating`, `bool IsVisibleToCamera`, `float SimTexelWorldArea`, `Bounds SimWorldBounds`, `bool IsWindowed`, `Vector3 SimWindowCenter`, `Vector3 SimWindowHalfExtent`, plus the runtime look/behaviour setters (`Foam`, `WindWaves`, `WaterFog`, `RippleStrength`, `RippleRadius`, `TimeScale`, `Quality`, `EnableCulling`, `IsPrimary`).

Uniform / MPB publishing path. Each frame `Update` calls `ApplyBodyBlock()`, filling `_mpb` via `WriteBodyProps` → `Publisher.WriteBodyProps(mpb)` and pushing the block to this body's own renderers (surface above/under, pool, god-ray, patch, clipmap, chunk shell) with `SetPropertyBlock`. `WaterUniformPublisher.WriteBodyUniforms(IUniformSink)` is the single derivation, fed either to `MpbUniformSink` (this body's block) or `GlobalUniformSink` (the primary's global mirror) so the two can never drift. `PublishSharedGlobals` pushes genuinely shared state (sun `_LightDir` / `_SunColor`, `_ScatterAmbient`, `_Tiles`, exclusion volumes). Notably `_WaveTime` and `_Sky` are **per-body** (never a shared global) to avoid last-writer-wins across bodies with

different clocks and skies. `WriteSimFrameUniforms(ComputeShader)` sets the volume + window frame onto a compute shader for GPU consumers.

`WaterBodyType` gating. `WaterBodyType { Pond, Lake, Ocean }` (serialized `bodyType`, default `Pond`) is **advisory only** — it drives the inspector UI and "Apply defaults", *not* runtime paths. Runtime feature gates read the actual booleans `openWater`, `unboundedOcean`, and `enableLargeBodyWindow` (`ShouldWindow`, `IsOceanClipmap`, the FFT module's `Enabled`). Migration v9 infers `bodyType` from those flags for legacy scenes.

Gotchas (from code comments).

- Module `Enabled` is evaluated once, before `Initialize`, per enable; `Dispose` must be safe when disabled / never-initialized.
- Per-body material/mesh instancing and pipeline-tier tweaks are **play-mode only** (an edit-mode instance could serialize a dead reference); originals are restored on `OnDisable`.
- `ApplyQuality` writes runtime `_`-prefixed fields, never the serialized `causticResolution`, so device-probed values are never baked into saved scenes.
- `WriteBodyProps` clears the block first, so any per-object look must live in the material, not the block.
- `WaterMembership.LateUpdate` calls `BodyContaining(pos)` then `body.WriteBodyProps(_mpb)`, so an object is lit by the lake it is actually inside; objects without it fall back to the primary's globals.

Settings & Tuning Surface

Overview. `WaterVolume.Settings.cs` is the serialized configuration half of `WaterVolume` (the runtime loop lives in `WaterVolume.cs`). It holds scene-wiring references, `[Header]`-grouped top-level fields, and eleven `[System.Serializable]` nested settings classes. Each was migrated out of the old flat class into a per-feature block; same-named internal forwarding accessors preserve every reader, and a versioned `OnAfterDeserialize` migration chain (`CurrentSettingsVersion = 9`) copies pre-migration values from hidden `_legacy…` fields exactly once. `[Header]` / `[Tooltip]` / `[Range]` / `[Min]` attributes map fields onto the inspector sections.

Top-level fields. Scene-builder wiring (`simCompute`, `oceanFftCompute`, caustics/obstacle/occluder shaders, `waterMesh`, `targetCamera`, `sun`, `tiles`, `sky`); `rippleQuality` (default `High`), `bodyType` (default `Pond`), `volumeExtent` (per-axis half-size); the large-water sim window (`enableLargeBodyWindow`, `largeBodyThreshold`, `simWindowMeters`, `clampWindowToShore`, `simWindowFocus`, `simWindowOffset`, `simWindowEdgeFadeTexels`); multi-instance (`surfaceAbove/Under`, `poolRenderer`, `godRayRenderer`, `isPrimary`, `autoLinkReceivers`); performance (`quality`, `enableCulling`, `activationDistance`); simulation (`timeScale`, `lightDir`, `causticResolution`); and Jerlov selection (`jerlovWaterType`).

Nested settings groups.

- **`OceanSettings`** — open-water / ocean look (inert on pools). `openWater`, `largeWaveAmplitude`, `largeWaveChoppiness` (horizontal Gerstner that sharpens crests), `swellHeight`, `swellWavelength`, `unboundedOcean`, `edgeFeatherMeters`; clipmap (`clipmapGridResolution`, `clipmapOuterRadius`, `oceanDetailFalloff`, `horizonFadeDistance`, `horizonHazeColor/Density`); ocean god rays (`largeGodRayColor/Density/Steps/Anisotropy/Extinction/CausticStrength`); whitecaps (`oceanFoamWindThreshold`, `oceanFoamCoverage/Strength/FadeRate/Color/TileSize/Feather/Deposit/Drift/MaxBuildup`).

- **`DetailNormalSettings`** — Crest-style crossing scrolling detail normals (micro-ripple finer than the FFT cascades): `texture`, `strength`, `tileMeters`, `scrollSpeed`.
- **`ReflectionSettings`** — `useScreenSpaceReflection`, `usePlanarReflection`, `reflectUrpProbe`, `realRefraction`; look (`reflectionStrength`, `envReflectionIntensity`, `fresnelFloor`, `fresnelPower`, `sunRoughness`, `roughnessFar/FarDistance/Falloff`, `reflectionAnisoStretch`, `sunSheen/SheenRoughness`, `sunGrazeBoost`, `reflectionDistortion`); SSR (`ssrStrength`, `ssrStepSize`, `ssrMaxSteps`, `ssrThickness`, `refractionDistortion`).
- **`ObjectInteractionSettings`** — `objectInteraction` (`MouseLikeDrops` / `FootprintDelta`), `obstacleStrength`, `obstacleDeadband` (swallows rasterization jitter), `obstacleSmoothing`, `obstacleFlipY`.
- **`WaterFogSettings`** — Beer–Lambert depth fog shared by surface / objects / pool: `waterFog`, `fogColor`, `fogExtinction` (HDR, red absorbs first), `fogDensity`, `waterOpacity`.
- **`VolumeScatterSettings`** — lit in-scatter over the fog: `volumeScatter`, `scatterColor`, `scatterIntensity`, `scatterAnisotropy`, `scatterAmbientTerm/SunTerm`; crest SSS (`crestScatter`, `sssIntensity`, `sssSunFalloff`, `sssPinchMin/Max/Falloff`).
- **`DepthAttenuationSettings`** — downwelling darkening independent of the view path: `depthDarken`, `depthExtinction`, `depthDarkenStrength`, `causticDepthFade`, `godRayDepthFade`, `linkDepthToFog`.
- **`BedDepthSettings`** — real terrain-bed depth plus shore / surf (the largest block): bed (`useBedDepth`, `bedTerrain`, `bedResolution`, `deepWaterColor`, `bedFadeDepth`, `bedTintStrength`); depth clarity (`clarityFromDepth`, `clarityShallow/DeepDepth`, `clarityShallow/Deep`, `clarityStrength`, `shoreShoalDepth`); shore transform (`shoreRefraction`, `shoreCompression`, `shoreGreens`); surf breakers (`surfEnabled`, `surfAmplitude`, `surfWavelength(Auto)`, `surfPeriod`, `surfBandDepth`, `surfSetStrength`, `surfCrestLength/Variation/Persistence`, `surfDirectionality`, `surfLean`, `surfAmbientFade`, `surfSwashAmplitude`, `surfFoamGain`, `surfWaterlineFoam`); surf-foam look and render-only enhancements (`surfCrestFoamCurve` baked to a 128-texel LUT, bore/trail/swash foam gains).
- **`WindWaveSettings`** — ambient wind layer: `windWaves`, `windSpeed`, `windFromDegrees`, `waveScaleMeters`, `waveCount`, `waveAmplitudeScale`, `waveDirectionSpread`, `waveNormalStrength`.
- **`FoamSettings`** — turbulence-driven pool/interactive foam (distinct from ocean whitecaps): generation (`foam`, `foamGenRate`, `foamDecay` — a *survival* factor, `foamDecayResidual/Rate`, `foamSpread`, `foamGenThreshold`, `foamMinWaveHeight`, `foamAdvect`, `foamFromSpeed/Curvature`, `foamDeposit`, `foamBreakStrength/Range`, `foamCrestBias`, `foamWakeStrength/RadiusScale`); shading (`foamColor`, `foamPatternSize`, `foamStrength`, `foamFeather`, `foamCoreCut`, `foamBorderWidth`, `foamContactDepth`).
- **`RippleSettings`** — the interactive solver: `waveSpeed` (stable to ~2.0), `damping`, `stepsPerFrame` (at a 60 fps reference), `rippleStrength`, `rippleRadius`, `rippleChoppiness`, `seedRipplesOnStart`, `conserveVolume`, `conserveMaxCorrection`.

Enums (defined in these files): `ReflectionMode { SkyOnly, SSR, Planar }` and `EnvironmentSource { ProceduralSky, UrpProbe }` are retired, kept only for the v7 migration onto the independent reflection toggles. `JerlovWaterType` (in `JerlovWaterTypes.cs`): `OceanI, OceanIA, OceanIB, OceanII, OceanIII, Coastal1C, Coastal3C, Coastal5C, Coastal7C, Coastal9C, Legacy` — clearest open ocean to most turbid coastal, plus a best-effort `Legacy` reproducing the pre-Jerlov tint.

Jerlov water types. `JerlovWaterTypes` is a static lookup of physically-derived colour presets. Each `JerlovPreset` carries `DisplayName`, `Extinction` (per-channel absorption in 1/m, feeding Fog Extinction at density 1) and `BodyColor` (single-scattering albedo, feeding the scatter/fog colour),

both linear-space, reconstructed from Solonenko & Mobley (2015) IOP fits integrated against the CIE 1931 colour-matching functions — data, not hand-tuned constants. `Get(JerlovWaterType)` indexes the ordered preset array. Selecting a type feeds one consistent physical source into both the see-through tint and the body glow; the inspector's "Apply water colour" button writes the fog-extinction / scatter fields.

Interactive Ripple Simulation

Overview. The interactive ripple module is a GPU height-field solver ported and expanded from Evan Wallace's WebGL water, running as a Unity 6 / URP compute pipeline. Each texel of a square grid stores `(height, velocity, normal.x, normal.z)`; an explicit wave-equation integrator propagates disturbances, and separate passes inject drops, moving-sphere wakes, mesh obstacle displacement, foam, and volume conservation. `WaterSimulation` owns the GPU state and dispatches; `WaterSimScheduler` budgets which bodies simulate; `WaterSimWindow` scrolls a camera-following window for large bodies; `WaterSurfaceSampler` + `WaterFieldSampling` mirror the field to the CPU for buoyancy; and `AsyncReadbackChannel` wraps `AsyncGPUReadback` with a mobile/WebGPU fallback.

`WaterSimulation` — data & stepping. `WaterSimulation : IDisposable` owns two `ARGBFloat` ping-pong RenderTextures `_a` (state) and `_b` (scratch), plus a `RFloat` foam ping-pong `_foamA` / `_foamB`. `Texture => _a` and `FoamTexture => _foamA` expose the live surfaces. `Resolution` (per side, per quality tier) must be a positive multiple of `ThreadGroupSize = 8`; `_groups = Resolution / 8`. RTs are `enableRandomWrite`, bilinear, Clamp, no mips, `HideAndDontSave`. Two `GraphicsBuffer`s back the exact-mean reduction. The core `Dispatch(kernel)` helper sets grid uniforms `_Size` / `_Delta`, binds `Src=_a` / `Dst=_b`, dispatches, then swaps so `_a` is always the latest state.

Stepping is frame-rate-independent: the caller runs `StepSimulation(waveSpeed, damping)` `stepsPerFrame` times, each `Update` dispatch binding `_WaveSpeed` (propagation stiffness, stable `0 < x ≤ 2.0`), `_Damping`, and `_WaveAxisWeight`. `SetAnisotropy` supplies per-axis Laplacian weights and drop squash so ripples read round in world on rectangular pools. `AddDrop` stamps height (floored to `MinDropTexelRadius = 2.5` texels). `AddSphereInteraction` injects a velocity-dipole wake *and* ping-pongs the foam buffer to stamp wake foam. `ApplyObstacle` / `SmoothObstacleFootprint` drive mesh displacement; `SetBedDepth` / `SetObstacleReflection` enable shoreline and reflector coupling on the Update kernel. `ConserveVolume(maxCorrection)` runs the two-pass reduction then `Conserve`.

`WaterSim.compute` kernels.

- **`Drop`** — stamps a smooth cosine height bump at `_Center` within `_Radius`, per-axis squashed; the classic mouse/touch splash.
- **`SphereInteract`** — accelerates the velocity channel with a directional dipole (a Crest `SphereWaterInteraction` port) so a moving object lays a Kelvin V-wake instead of rings; also stamps wake foam. Copy-through outside the cull radius.
- **`Update`** — the wave-equation integrator: per-axis weighted Laplacian restoring force → velocity → damping → height. Handles bed-depth shoreline (dry land held flat so ripples reflect; open-shore drain band) and static reflector walls (Neumann boundary).
- **`Normal`** — rederives the surface normal from height gradients into `.ba`.
- **`Obstacle`** — pushes the surface by the blurred (5×5 Gaussian) delta of the submerged footprint with a soft dead-band; generalises sphere displacement to arbitrary meshes.

- **`ObstacleSmooth`** — temporal EMA of the raw footprint into an `RFloat` target, low-passing bob/rotation noise (in compute because the fullscreen equivalent failed on WebGPU).
- **`Foam`** — semi-Lagrangian advection of foam along surface flow, isotropic diffusion, generation from coherent speed/curvature/shear with breaking and amplitude gates, shallow-water breaking boost, surf-front injection, deposit floor, and bi-exponential decay.
- **`ReduceMean`** / **`ReduceMeanFinal`** — the exact group-shared volume reduction (pass 1 per 8×8 group; pass 2 a single group).
- **`Conserve`** — subtracts the exact mean height (clamped to `±_MeanCorrectionMax`) so a closed pool conserves volume.
- **`Scroll`** / **`ScrollFoam`** — integer-texel shift of the state / foam buffers so ripples stay world-anchored as the sim window follows the camera.

Kernel names are validated up front against `RequiredKernels`, so a stale compute asset fails with one clear message.

Scheduling & sim window. `WaterSimScheduler` is a static, frame-guarded scheduler. In play mode, pass 1 sets `_visible` by frustum test on `CullBounds()` and a provisional `_simulate` for visible, in-range bodies; pass 2 (`EnforceSimBudget`) caps the simulating set to the nearest `ActiveSimBudget = 4`. `WaterSimWindow` owns a texel-snapped world `Center` for large bodies: `Track()` projects the follow target onto the surface, adds an optional lead/lateral offset, clamps into the footprint (unless an ocean clipmap, which scrolls freely), snaps to the texel lattice, and when the cell index changes calls `sim.Scroll(-dx, -dz)` so world features stay fixed.

CPU read-back & fallback. `AsyncReadbackChannel` wraps `AsyncGPUReadback` with a single-in-flight throttle. It issues a mip-0 read-back only when idle and supported, and latches `Unsupported` after `MaxConsecutiveErrors = 8` failures or at construction on backends without `SystemInfo.supportsAsyncGPUReadback` (WebGPU/mobile). When unsupported, callers serve queries from the analytic waterline (flat rest + wind waves), so buoyancy keeps working — only interactive ripples and obstacle displacement are absent there. `WaterSurfaceSampler` owns the height read-back into a reused `Color[]`, composites the surface sample, and adds analytic wind-wave detail; `excludeRipples` serves a self-emitting floater the analytic surface only, breaking the self-propulsion feedback loop. `WaterFieldSampling` is the single shared bilinear filter (half-texel offset, edge-clamped) used by the ripple, ocean-FFT, and shore-depth CPU mirrors so they can never drift.

Gotchas (from comments).

- WebGPU allows only `r32` read-write storage, so `Src` is a sampled `Texture2D` and `Dst` a write-only `RWTexture2D`; foam/obstacle targets are `RFloat`; foam advection does manual bilinear because `r32` isn't filterable.
- Volume conservation was rewritten: the old mip-based "mean" silently point-sampled on WebGPU and could subtract a wave crest and pop the whole plane; the exact group-shared reduction replaced it.
- `Sanitize` resets non-finite texels and clamps to `MAX_ABS_HEIGHT = 1.0` / `MAX_ABS_VELOCITY = 0.5` to stop one bad value poisoning the plane.
- The reduction kernels keep every `GroupMemoryBarrierWithGroupSync` in straight-line control flow with `select()` because WGSL rejects the classic in-loop tree reduction.
- Bed, solid, and shore textures are always bound (black fallback) so the WebGPU backend never sees an unbound sampler; `_UseBedDepth` / `_ObstacleReflect` / `_ShoreFoamActive` gate every read.

Ambient Waves & Ocean (Wind Waves, Swell, FFT Ocean)

The stacked height-sources model. On top of the interactive ripple sim, the package composites several independent, world-space wave sources. Each is a pure function of position + time (so the CPU physics can mirror the GPU render with no read-back), and they are *summed* into the surface height/normal. Coarsest interface first:

1. **Wind-wave sum-of-sines bank** — `WaterWaveBank` + `WaterWaves.hlsl`, pool-space, used by small bounded bodies.
2. **Analytic large-wave field** — `WaterLargeWaves.hlsl` (Gerstner chop + swell bands) with its byte-for-byte CPU mirror `LargeWaveField`, plus the shore/surf layer in `WaterSurfWaves.hlsl`.
3. **FFT / Tessendorf ocean cascade** — `WaterOceanFft` + `OceanFft.compute`, which *replaces the body of the large-wave height/displacement path* behind a stable interface (`_OceanFftActive`).
4. **Ocean horizon clipmap** — `LargeWaterClipmap` + `WaterVolume.OceanClipmap.cs`, the world-locked mesh the ocean draws on.

Which source applies is gated by configuration: pools/lakes use the wind bank or the analytic large-wave path; only an opted-in ocean (`IsOceanClipmap`) builds the clipmap and runs the FFT.

Wind-wave bank. `WaterWaveBank.Generate(...)` builds up to `MaxWaves = 16` directional sinusoids from a **JONSWAP-shaped** spectrum (`Jonswap()` = Pierson–Moskowitz base × peak enhancement `JonswapGamma = 3.3` with asymmetric sigmas). The peak wavelength is clamped to `[0.6, 6]` m. Components are stratified across a wide fan with a golden-ratio sequence, with a fraction facing upwind at reduced amplitude, so crests cross into choppy, non-directional lake water. `NormalizeAmplitudes` scales the set so significant height tracks wind (fetch-limited), with a scale-independent metre→pool conversion. Deep-water dispersion `omega = sqrt(g·k)` throughout. Results pack into `_WaveA` / `_WaveB` / `_WaveCount`; `WaterWaves.hlsl` evaluates the *identical* sum, and the CPU side offers `SampleHeight`, `SampleSlope`, and `SampleVerticalVelocity`. A `minWavelengthMeters` LOD filter lets large floaters ignore fine ripples. This layer is vertical-only (no Gerstner pinch), keeping height a true function of (x,z).

Analytic large waves (swell + chop). `WaterLargeWaves.hlsl` is the world-space open-water field. `EvaluateLargeBodyWaveShore` sums two directional Gerstner bands via `LbwAccumulateBand`: a wind **chop** band (`LBW_WAVE_COUNT 12`, `LBW_BASE_WAVELENGTH 9`) and a long-period **swell** band (`LBW_SWELL_COUNT 4`, driven by `_LargeSwellWavelength` / `_LargeSwellHeight`). It returns a `LargeBodyWaveField` (height, slope, horizontal `disp`, and `dispDeriv` for the Jacobian normal). `_LargeWaveChoppiness` scales the horizontal Gerstner offset; `LbwEdgeWeight` feathers the composite to zero at a bounded footprint's border. `WaterSurfWaves.hlsl` layers **shore-parallel breaker fronts** on top: a pure closed-form lifecycle with Green's-law growth, Iribarren breaker classification (spill/plunge/surge), slope-dependent gamma, lean, and a Dally–Dean–Dalrymple bore — everything derived from a *wrapped* front index on the master `_SurfBeatTime` clock so float32 `sin()` never drifts. `LargeWaveField.cs` is the **byte-for-byte CPU mirror** of both, re-implementing the accumulation, the shore transform, and a 4-iteration Gerstner-chop inversion so it can return the surface directly above a fixed world point for buoyancy.

FFT ocean cascade. `WaterOceanFft` (ocean-only, constructed when `IsOceanClipmap`) owns the Tessendorf pipeline in `OceanFft.compute`. Defaults: `DefaultResolution 128`, `DefaultCascadeCount`

`4`, ascending `DefaultDomainSizes {5, 20, 100, 600}` m so each cascade owns a disjoint band. Real kernels:

- **`SpectrumInit`** — static H0 from `OceanPhillips` plus a narrow travelling-swell peak; rebuilt only on wind/swell change.
- **`SpectrumUpdate`** — evolves H0 to `OceanFftTime` into three complex spectra (X, height Y, Z).
- **`FftHorizontal` / `FftVertical`** — in-place group-shared butterfly inverse FFT (row then column, precomputed twiddles), with the fftshift and per-cascade height scales, writing `OceanDisplacement`.
- **`ComputeNormal`** — normals + Jacobian foam. Writes `OceanNormal` under a channel contract: `.xz` = tilt, `.y` = crest pinch `saturate(1−J)`, `.w` = accumulated whitecap foam (frame-rate-independent split decay + downwind advection, ping-ponged history).
- **`BakeHeightField`** — sums cascade heights over a 256 m camera-centred region into `OceanHeightField` for CPU buoyancy.
- **`VisualizePreview`** — a debug remap (editor/dev only).

`Dispatch(...)` publishes globals (`_OceanFftDisplacement`, `_OceanFftNormal`, `_OceanFftDomainSizes`, …) that `WaterLargeWaves.hlsl` samples when `_OceanFftActive > 0.5`. `RequestHeightReadback` / `OnHeightReadback` do throttled async read-back; `TrySampleField` / `TrySampleHeightLatest` serve buoyancy and the fog gate from the landed CPU copy.

Ocean horizon clipmap. `LargeWaterClipmap.BuildAnnulusTemplate` builds one shared uniform square-annulus grid in integer cell units (a Losasso/Hoppe geometry clipmap). `WaterVolume.OceanClipmap.cs` draws it as nested LOD levels; each `ClipmapLevel` scales the template by `cellSize·2^L` and snaps its centre to that level's own world lattice, so vertices re-land on a fixed world lattice and never "swim" under the world-space wave field. Above/under twins, per-level geomorph, and depth bias stitch levels crack-free. `CreateOceanClipmap` runs **play-mode only**, gated on `openWater && unboundedOcean && !_windowed` and `IsOceanClipmap`; bounded lakes/pools never build one.

Why the mirror constraint matters. Buoyancy must sample the *same* surface the eye sees without a per-frame GPU read-back for the analytic path — so `WaterWaveBank ↔ WaterWaves.hlsl` and `LargeWaveField ↔ WaterLargeWaves.hlsl / WaterSurfWaves.hlsl` duplicate their math and constants exactly. If a constant, hash, or wrap changes on one side and not the other, floaters silently desync from the rendered crest — hence `WaterWaveConstantsValidator`. The FFT ocean escapes the byte-for-byte rule differently: its height *is* GPU-authored, so buoyancy reads the baked `OceanHeightField` via async read-back (the same field the shader renders).

Gotchas. `_OceanFftActive` swaps the large-wave height/normal/foam body from analytic Gerstner to cascade lookups without changing call sites; on unsupported devices the ocean falls back to the analytic path cleanly. The surf layer *replaces* ambient wave energy where it owns the surface (anti-double-crest). Every surf term runs on the wrapped `_SurfBeatTime`, never raw `_WaveTime`. FFT buoyancy has deferred limitations: no choppiness inversion, a finite 256 m read-back region, and 1–2 frames of async lag.

Buoyancy, Physics & Interaction

Overview — two-way coupling. This module closes the loop between the GPU water sim and the rigidbodies on it. The **water → object** half reads the surface and pushes objects (`WaterBuoyancy`, `BoatController`); the **object → water** half stamps the surface

(`WaterInteractable` , `WaterObstacle` , `WaterSphereInteractor` , `WaterRippleEmitter`). Both talk to the surface only through world-space façades on `WaterVolume` and the batched height seam (`IWaterHeightSampler`), so nothing reaches into sim internals, and any body an object drifts over is resolved per-frame via `WaterVolume.BodyContaining` .

`WaterBuoyancy` (`[RequireComponent(typeof(Rigidbody))]` ; add beside a `Collider`). Rather than one force at the centre of mass (which can only bob), it lays a lattice of float points across the collider's local bounding box (`samplesPerAxis` 1–3 → n³ points; 2 = 8 corners for good torque) and applies a partial buoyant force at *each* point, so the summed forces yield righting torque, roll, and pitch. Each `FixedUpdate` it re-resolves the body, transforms points to world, and issues **one** batched `SampleHeights(...)` . Results are rented per-owner from a shared `WaterHeightQuery` (keyed on `GetInstanceID()`), so there is no per-frame allocation. For each valid sample it computes depth below the surface along `VolumeUp` , converts to a submerged fraction via a spherical-cap S-curve, then `ApplyPointForces` : Archimedes lift (`ForceMode.Acceleration` , optionally clamped by `maxBuoyancyForce`), per-point drag, and a wave-drift push along the surface velocity. Key fields: `buoyancy` , `waterLinearDamping` , `waterAngularDamping` , `samplesPerAxis` , `floatRadiusScale` , `waveDriftStrength` , `verticalSettleDamping` . Opt-in extensions (defaults reproduce the original single-point model): `objectWidth` (ripple LOD cut-off), `maxBuoyancyForce` , `surfaceRelativeDrag` + `dragCoefficients` (anisotropic brake toward the water's own velocity), `ignoreInteractiveRipples` (sample the analytic surface only — for a body emitting its own wake), and `drawDebugGizmos` . When read-back is unavailable the query path is CPU-analytic, so samples are valid from frame 0. `WaterBuoyancyStressSpawner` spawns a parametric grid of cubes to profile the probe budget.

`WaterInteractable` **&** **`WaterObstacle`** **.** `WaterInteractable` (`[RequireComponent(typeof(Renderer))]`) marks any Renderer that disturbs the surface and self-registers into a static `Active` list. Field `displaceScale` weights it; in **MouseLikeDrops** mode its `LateUpdate` emits analytic cosine drops cloned from the mouse rules (one per `verticalEmitSpacing` of plunge/rise, one wake drop per `horizontalEmitSpacing` of travel, capped at `MaxDropsPerFrame = 4`). Plunge depth is tracked against the *analytic* waterline, deliberately not the live readback (which would feed the object's own stale ripples back into its footprint). Public API: `float WaterlineY(float restY)` , `bool IsSubmerged(float waterlineY)` , `Renderer Renderer { get; }` , static `IReadOnlyList<WaterInteractable> Active` . `WaterObstacle` (a plain C# class) is the **FootprintDelta** path: it rasterizes every interactable top-down into a ping-pong pair of `RFloat` RTs via `ObstacleDepth.shader` , and the compute sim reads `prev − curr` to push the surface — generalising the analytic sphere kernel to arbitrary meshes. The footprint is drawn at 4× supersample then box-filtered, so an edge sweeping a texel becomes a smooth coverage ramp instead of popping micro-ripples. API: `SetFrame` , `Render(waterY)` , `RenderSolid(waterY)` (reflector-only mask), `Dispose()` , and RT accessors `Prev/Curr/Raw/Solid` .

`WaterSphereInteractor` **.** Unlike the isotropic height rings above, this injects a **directional velocity dipole** (pushed ahead of travel, pulled behind — a Crest-style V-wake). Each `LateUpdate` it measures its displacement and forwards it via `WaterVolume.TrySphereInteractionAt(...)` . Fields: `centerOffset` , `radius` (0 = auto from bounds), `strength` , `maxStepDistance` (teleport guard), and `wakeFadeSpeed` (a low-speed taper so a boat slowing to a creep sheds its wake). Composition-only, so it coexists with `WaterBuoyancy` and `BoatController` on one rigidbody.

Input routing. `WaterInputRouter` (owned only by the primary body, play-mode only) routes pointer input each frame: it raycasts against every body's surface and, on a hit, injects ripples through `AddRipple` throttled by world travel (so holding still can't pump energy), plus drag-

splashes scaled by cursor speed; clicking empty space orbits the camera. Keys: **Space** toggles pause, **L** aligns the sun to the camera. On touch, only a *tap* fires a ripple; two or more fingers yield to pinch-zoom. Input is abstracted over the new Input System with a legacy fallback. `WaterTouchInput` holds the shared touch mechanics (a `PinchTracker` struct, two-touch helpers).

Height-query seam. `IWaterHeightSampler` is the allocation-free façade `WaterVolume` implements: `bool SampleHeight(Vector3, out WaterSample, float minimumLength = 0, bool excludeInteractiveRipples = false)` and the batched `void SampleHeights(int ownerHash, float minimumLength, IReadOnlyList<Vector3>, WaterSample[], WaterQueryFields = HeightNormalVelocity, bool = false)`. `WaterSample` is `{ float Height, Vector3 Normal, Vector3 Velocity, bool Valid }` — `Valid` is false outside the footprint. `WaterQueryFields` is a `[Flags]` enum (`Height` / `Normal` / `Velocity`) so unused fields skip their per-point work. The implementation is CPU-analytic (shared wave mirrors), returning valid data from frame 0 with no async read-back. `WaterHeightQuery` is the per-owner reusable-buffer registrar (`RentResults`, `Release`).

Helpers & the boat example. `WaterRippleEmitter` spawns ripples into whichever body contains its emit point (`Emit()` on demand, or `emitOnMove` for a continuous wake). `WaterProbe` watches one point and fires `Submerged` / `Emerged` `UnityEvent`s. `BoatController` (`[RequireComponent(typeof(Rigidbody))]`) is the example consumer: it supplies only what buoyancy can't — thrust at the stern eased by propeller wetness, speed-scaled steering (no authority at rest), and quadratic keel + lateral drag that carve turns; float/roll/pitch come from `WaterBuoyancy` probes on the same object.

Gotchas. The whole query seam is CPU-analytic precisely because async GPU read-back is unavailable on the deployed WebGPU build. A body that emits its own wake (a boat) must set `ignoreInteractiveRipples` or it gets pushed by its own ripples via a stale-readback feedback loop. The obstacle supersample chain is `RHalf` (WebGPU can't blend into float32 on many mobile GPUs) while `prev/curr/raw` are `RFloat`; the smoothing EMA runs as a separate compute kernel because a fullscreen material pass for it silently failed on this backend.

Foam & Particles

The module renders three distinct whitewater elements that share one look (`WaterFoamCommon.hlsl`) but ride different tech paths: a **surface foam field** (baked by the sim, shaded on the surface), **GPU foam/spray particles** (`WaterFoamParticles`), and **impact splashes** (`WaterSplash` / `WaterSplashEmitter`).

Surface foam field. The field is produced by the sim compute passes — `OceanFft.compute` accumulates whitecap foam, `WaterSim.compute` accumulates ripple-sim foam. Both decay through the shared `FoamDecayBlend(...)` — a bi-exponential ("split") decay blended by local foam amount — but with deliberately **opposite semantics**: `OceanFft.compute` blends decay *rates* so dense deposited foam decays **slower** (windrows linger), while `WaterSim.compute` blends *survival* factors so thick fresh foam decays **faster** (bright collapse, then thin lace lingers). A single exponential can't do both. Shading uses `WaterFoamCommon.hlsl` helpers shared across every foam element: wrapped diffuse (`FOAM_LIGHT_WRAP 0.4` so unlit foam never goes black), `FoamLitColor`, and erosion dissolve — `FoamErosionAlpha` (a pure gate) and `FoamErosionLace` (texture-preserving, so a sprite's own lace shows fresh and crumbles as its envelope decays).

GPU foam particles (`WaterFoamParticles`). `[AddComponentMenu("AbstractOcclusion/Water/Water Foam Particles")]`, attached beside a `WaterVolume`. The pipeline is WebGPU-safe by construction: no Append/Consume buffers (a ring cursor advanced with `InterlockedAdd`), no CPU

read-back, procedural quads pulled by `SV_VertexID` (6 verts/particle). `OnEnable` degrades gracefully — if vertex-stage structured buffers are unsupported it disables; density mode needs two fragment-stage buffers or falls back to quads. **`FoamRenderMode`** is `ScreenSpaceDensity` (KWS-style: particles splat into a screen-space density buffer, a fullscreen composite shades connected lit foam) or `Quads` (per-particle billboards; also the automatic fallback). Spray droplets are always billboards.

Compute kernels in `WaterFoamParticles.compute`: `BeginFrame` (resets per-frame counters and the 16×16 spray-tile budget), `Spawn` (one thread per sim texel; rolls a spawn where the foam field exceeds `_SpawnThreshold`, with world-lattice-keyed rolls, stochastic distance LOD, shear boost, per-tile spray budget, and surf plunging-lip jets; emits `KIND_SURFACE` or `KIND_SPRAY`), `SpawnBurst` (one thread group per CPU-queued burst), `Update` (integrate, age, edge-fade + hard kill outside the sim frame, landed-spray → surface foam, curl+clump drift, wind, dry-interior kills), `ClearDensity`, and `RasterizeDensity` (live particles splat life-enveloped weight and eye depth into the screen-space buffers). The density splat is deferred to `OnBeginCameraRendering` so it projects with the camera's final frame matrices. Key fields: `capacity` (tier-capped, pow2), `spawnThreshold`, `spawnRate`, `maxSpawnPerFrame`, `sprayChance`, `sprayLaunchSpeed`, `lifeRange`, `sizeRange`, `sizeHeroPower` (rare "hero" clumps), `spawnMaxDistance`, `gravity`, `flowDrift`, `windDriftSpeed`, `drag`, `crestRollSpeed`, `flipbookGrid`, `flipbookFps`. The `FoamParticle` struct is 48 bytes, matched across C#, compute, and shader.

`WaterFoamProfile` & `WaterParticlePool`. `WaterFoamProfile` is one master `ScriptableObject` bundling four sections, each with a `drive` toggle: `SharedLook` (tint, opacity, atlas, flipbook grid/fps, `sizeHeroPower`), `AmbientSection` (spawn/spray/life/size fields), `VeilSection` (density composite: opacity, low/high gains, breakup lace), and `SplashSection` (burst/crown shaping). A `null` profile means each component keeps its own inspector values. Driven fields are copied onto components every frame/emit; shared look and veil values are pushed over materials via `MaterialPropertyBlock` so `.mat` assets are never written. `WaterParticlePool` is a static helper that builds the tier-capped, pow2 pool (dead slots `life = 0`) plus a counters buffer.

Splashes. `WaterSplash` (`[RequireComponent(typeof(Rigidbody))]`) detects a body punching through the live waterline in `FixedUpdate` and calls `emitter.EmitSplash(...)` plus `AddRipple(...)`. `WaterSplashEmitter` owns a Shuriken system for drift droplets and an optional crown; its `LateUpdate` implements pop → stick → drift (droplets launch ballistically, snap to the surface, then ride the CPU-read local flow). Crucially, `EmitSplash` **unifies airborne spray**: when the body has an active `WaterFoamParticles`, it maps burst shaping onto `gpuSpray.QueueSplashBurst(...)` so event droplets share the exact `KIND_SPRAY` tech/look, and Shuriken plays only the crown flipbook. Bodies without a GPU system keep the legacy Shuriken burst. `QueueSplashBurst` is a soft-budget queue (≤16 bursts/frame, ≤64 droplets each).

Shaders & gotchas. `FoamParticles.shader` (Transparent+10) pulls each particle by `SV_VertexID`, glues surface foam into the animated plane and billboards spray stretched along velocity. `FoamDensityComposite.shader` (Transparent+5) reads the density buffer with a 4-neighbour MAX dilation, maps density → foam via a two-term curve, writes splatted min depth to `SV_Depth` for exact per-pixel wave occlusion, and adds world-anchored breakup lace. `SplashParticles.shader` shades Shuriken crown/droplets with the same foam lighting. Foam density scales per **quality tier** (`WaterParticlePool.Allocate` caps `capacity` at `volume.FoamParticleBudget`), so weak devices pay for fewer live particles regardless of the inspector value. The `FoamDecayBlend` opposite-semantics contract is the key correctness note. Burst-jitter constants are mirrored between `WaterSplashEmitter.cs` and the compute and guarded by `WaterWaveConstantsValidator`.

Rendering, Optics & Quality

Overview. The optics stack is split between per-body work driven from C# (`WaterCausticsPass` , `PlanarMirror`) and fullscreen effects injected as URP `ScriptableRendererFeature` s under `Runtime/Rendering/` . Every feature self-gates: it holds a serialized shader, builds an engine material in `Create()` , and in `AddRenderPasses` early-returns unless a static flag says the effect is live this frame — so a scene with no ocean, no chunk, and a dry camera pays nothing. All three fullscreen features are guarded by `#if WEBGPUWATER_URP` and use the RenderGraph API.

Caustics. `WaterCausticsPass` is owned per body. It builds materials from `Caustics.shader` , `LargeBodyCaustics.shader` , and `CausticOccluder.shader` , plus an `ARGB32` RT (`CausticTex`) and a `CommandBuffer` . `Render()` clears the RT to `(0,1,0,0)` — green starts at 1 (unshadowed) — then draws the body's water grid; the caustics vertex stage projects each vertex along the refracted sun onto the pool floor and outputs clip space directly (compensating the render-target Y-flip via `_ProjectionParams.x`), and the fragment writes light focusing as the ratio of old/new projected triangle area into red. Because each body renders its own sim into its own RT, caustics never leak from whatever body last wrote the `_WaterTex` global. `DrawOccluders` then stamps only this body's submerged interactables into green via `CausticOccluder.shader` , walking down the refracted ray to the floor in pool space so the object's underwater shadow lands *with* the caustics rather than where the un-refracted shadow map puts it. `RenderLargeBody` is the ocean path (`LargeBodyCaustics.shader`) rebuilding the same Jacobian focusing in the moving sim window's world frame.

God rays. Two shaders produce real shadow shafts. `GodRays.shader` is a pool-box additive volume (`Blend One One` , `Cull Front` , `Queue Transparent+100`) marched in pool space; each of `_GodRaySteps` samples reads the projected caustic, multiplies by a shadow term (the occluder green when active, else `MainLightRealtimeShadow`), applies exclusion visibility for dry carved rooms, and thins by depth-fade and view-fog exponentials. **This pass requires "Transparent Receive Shadows" enabled on the URP asset** — URP only feeds the main-light shadow to transparent passes with that toggle on; with it off, `MainLightRealtimeShadow` returns "lit" and the shafts silently vanish. `LargeBodyGodRays.shader` is the ocean fullscreen analog (half-res raymarch with a Henyey–Greenstein phase, then additive composite), driven by `LargeBodyAtmosphereFeature` / `Pass` , gated by `LargeBodyAtmosphereGate.HasActiveGodRayOcean` , injected at `BeforeRenderingPostProcessing` so bloom/tonemapping treat the shafts as scene light.

Reflections. Reflection mode is uniform-driven and published per body every frame — no keywords, no per-body material instancing. `PlanarMirror` renders a second mirrored camera across `y = waterHeight` into an owned HDR RT (with a trilinear mip chain so roughness blurs the mirror) using URP's `SingleCameraRequest` , guarding culling inversion in a `try/finally` . `WaterVolume.Underwater.cs` drives planar per body, each pool mirroring its own plane. Planar is the only mode with real per-frame cost — each mirror is a full extra scene render — so `WaterReflections` caps it: `MaxActivePlanarBodies = 3` , recomputed once per frame, keeping the nearest to camera and degrading the rest to SSR/sky. Tiers gate the richer modes via `RichReflections` (off → every body caps to SkyOnly) and `RealRefraction` (off releases the URP opaque-texture copy, falling back to the analytic pool look).

Underwater. `WaterVolume.Underwater.cs` arms the fog gate at `OnBeginCameraRender` . `ComputeCameraSubmerged` tests the near-plane corners against a wave-aware surface height with hysteresis to stop crest toggling. `WaterUnderwaterFogFeature` / `Pass` then runs two hardware-blend fullscreen passes at `BeforeRenderingPostProcessing` — absorb (`Blend Zero SrcColor`) then

inscatter (`Blend One One`) — reading `cameraColor` as `ReadWrite` so no scene copy is needed. `WaterUnderwaterFog.shader` computes the in-water path length per pixel (ocean half-space or pond box via `IntersectCube`) using per-channel Beer–Lambert, downwelling attenuation, exclusion carve-outs, and an inscatter colour continuous with the surface. `WaterWaterline.hlsl` is the single source of truth for `SurfaceHeightAtXZ` / `SurfaceSignedGap` , shared with the exclusion wall so both clip at the same meniscus.

URP renderer features / passes.

Feature	Pass	What it does / where
`LargeBodyAtmosphereFeature`	`LargeBodyAtmospherePass`	Ocean god-ray shafts: half-res raymarch + additive composite at `BeforeRenderingPostProcessing` . Gated on `HasActiveGodRayOcean` .
`WaterChunkDepthFeature`	`WaterChunkDepthPass`	Mesh-chunk depth prepass: front faces → `_ChunkFogFrontDepth` , back → `_ChunkFogBackDepth` , at `BeforeRenderingTransparents` , so the chunk wall bounds its column against an arbitrary closed mesh. Gated on `AnyMeshChunkActive` .
`WaterUnderwaterFogFeature`	`WaterUnderwaterFogPass`	Fullscreen Beer–Lambert fog (absorb + inscatter) at `BeforeRenderingPostProcessing` . Gated on `UnderwaterFogActive` .

`WaterReceiver` . `WaterReceiver.shader` is a full URP lit surface for floors/props: `ForwardLit` , `ShadowCaster` , `DepthOnly` , and a `DepthNormals` pass (so the receiver populates `_CameraDepthTexture` under an SSAO depth-normals prepass — otherwise god rays drew over the floor). It gates water shading on a pool footprint mask, measures depth against the sampled sim surface (manual bilinear because WebGPU can't filter float32), applies downwelling, projects caustics with `SAMPLE_TEXTURE2D_GRAD` (implicit derivatives are undefined in a non-uniform branch under WGSL), and takes the object shadow from caustic green when active.

Quality tiers. `WaterQuality` is a `ScriptableObject` with `selection` (`Auto` / `ForceLow` / `ForceMedium` / `ForceHigh`). `Resolve()` returns a forced tier or `Probe()` , which forces Low on WebGL/mobile/no-async-readback, Medium under a VRAM threshold, else High. Each tier is an immutable `Tier` struct scaling: `SimResolution` (256/128/128), `CausticResolution` (1024/512/256), `GodRaySteps` (24/16/12 — god rays kept **on** at Low), `RichReflections` (on/on/off), `MaxWaveCount` (32/12/8), `RefineSteps` (5/3/2), `RenderScale` (1/1/0.7), `RealRefraction` (on/on/off), `MeshDetail` (0/0/100), `CausticInterval` (1/1/2), `ReadbackInterval` (1/1/3), `MaxFoamParticles` (65536/65536/1024), and `UnderwaterFog` mode (Full/Full/Simple). `WaterMetricsOverlay` is a debug HUD showing smoothed frame ms/FPS, live floater count, and the batched buoyancy-query cost read via a `ProfilerRecorder` on the `SampleHeights` marker.

Gotchas. *Transparent Receive Shadows* must be on or god rays lose all shafts silently; the passes need *Depth Texture* on. Each planar body is a full extra scene render — the `MaxActivePlanarBodies = 3` budget is the cost lever. The caustic RT green defaults to 1 (lit); the large-body path clears it to 0. Fog/god-ray passes bind `cameraColor` as `ReadWrite` (not `Write` , which would render black). Raw clip-space caustic output must honour `_ProjectionParams.x` or the map mirrors on flipped backends.

The Water Surface Shader

Overview. `WaterSurface.shader` (`AbstractOcclusion/WebGpuWater/WaterSurface`) is the one material that renders the visible surface. The scene builder instances it twice — an "above water" object (`_Underwater = 0`, `Cull Front`) and an "under water" object (`_Underwater = 1`, `Cull Back`) sharing the same displaced grid. From that one pass it composites a large stack of keyword- and uniform-gated effects: hybrid reflection (analytic sky/probe → planar → SSR), refraction (analytic pool ray-trace or real screen-space), Fresnel + dual-lobe GGX sun specular, several foam systems, shoreline/surf swash, subsurface crest glow, detail normals, horizon haze, and exclusion/chunk clipping. Defaults keep unset features inert so the base look matches the original WebGL demo.

Structure. Tags: `"RenderType"="Transparent" "Queue"="Transparent" "RenderPipeline"="UniversalPipeline"` — the transparent queue is required so `_CameraOpaqueTexture` / `_CameraDepthTexture` hold the scene *without* the water (needed for SSR and screen-space refraction). Render state: `Cull [_Cull]`, `ZWrite On`, `ZTest LEqual`, no blend (the shader computes the final opaque-looking colour itself). It is a single `CGPROGRAM` pass (`#pragma target 4.0`), UnityCG-style rather than full SRP: it `#include "UnityCG.cginc"` and uses `sampler2D` / `tex2D`, `fixed4`, single-arg `LinearEyeDepth`, and `_ProjectionParams.z` — so it *cannot* include URP's `Shadows.hlsl`, and samples the shadow map by hand.

The **vertex stage** feeds three sources into the same ripple/wave path — full plane (grid vertex is pool xz), window patch (`_IsPatch`, `[-1,1]` remapped into a camera-following sub-region), and ocean clipmap (`_IsClipmap`, world-metre cells with even-lattice edge geomorph to avoid T-junction cracks). Height is layered: interactive ripple field (`SampleRipple`, windowed & edge-faded), then `WaveHeight` wind-wave detail, then for `_LargeBody` the world-space swell + Gerstner choppiness, plus interactive-ripple horizontal choppiness (`_RippleChoppiness`) and the surf swash film. The **fragment** does an early exclusion `discard`, chunk footprint clips, then staged shading: `EvaluateSurfaceGeometry` → `EvaluateWaterClarity` → either `UnderwaterStage` or the above-water chain (`ReflectionStage`, `RefractionStage`, `EvaluateCrestGlow`, `FoamLayersStage`, `CompositeSurfaceColor`, `ApplyShallowClarity`, `ShorelineStage`, `FinalCompositeStage`).

Include-file breakdown.

- **`WaterSurfaceScreen.hlsl`** — URP screen-texture access (`_CameraOpaqueTexture`, `_CameraDepthTexture` with a shared point-clamp sampler), `RawSceneDepth`, `ScreenUV`, `EyeDepthOf`.
- **`WaterSurfaceShadow.hlsl`** — the hand-rolled URP main-light shadow tap (cascade select + one hard depth compare, honouring `UNITY_REVERSED_Z`), gated by the `_MAIN_LIGHT_SHADOWS*` keywords; used to gate the analytic floor caustic.
- **`WaterSurfaceSpecular.hlsl`** — the specular/reflection family: Schlick Fresnel from IOR, a smoothness-far roughness ramp, sky-environment sampling (sharp/rough/anisotropic), planar sampling, SSR march + smeared opaque sampling, and dual-lobe GGX sun specular.
- **`WaterSurfacePoolTrace.hlsl`** — the analytic pool ray-trace: refract a down ray into the pool box to sample the tile floor/walls, environment for an up ray, and `DeepWaterColor` in-scatter for open/large bodies with no floor.
- **`WaterSurfaceFoamSampling.hlsl`** — every foam *sampling* path (mask bilinear, flipbook frames, ocean-whitecap pattern with anti-tiling second octave, the shared `FoamDissolve` and `ApplyFoamTiltToNormal`). Foam *lighting* lives in `WaterFoamCommon.hlsl`.

- **`WaterSurfaceDetailNormal.hlsl`** — Crest-style crossing scrolling detail normals sampled at two world scales along two fixed directions, faded by camera distance. Inert with the default `"bump"` texture.
- **`WaterSurfaceFragStages.hlsl`** — the master fragment assembly; must be the *last* include (directly above `frag`), since it reads the pass-local uniforms, `v2f`, `SampleRipple`, and `vert`.

Shared helpers. `WaterShared.hlsl` (backend-agnostic constants + pure math, no samplers: IOR, pool-box constants, `IntersectCube`, `ProjectCausticUV`); `WaterCommon.hlsl` (global sampler declarations `_WaterTex` / `_CausticTex` / `_Tiles` / `_Sky`, manual bilinear/bicubic sim sampling, wall shading); `WaterVolume.hlsl` (the placement frame: `PoolToWorld` / `WorldToPool`, the sim-window frame, footprint masking).

Keywords vs uniforms. The *only* variant multi_compile is `_ _MAIN_LIGHT_SHADOWS _MAIN_LIGHT_SHADOWS_CASCADE` (gating the shadow tap). **Almost everything else is uniform-driven, not keyword-gated** — published per body via the `MaterialPropertyBlock`, so it updates live with no variant explosion: reflection (`_UsePlanar`, `_UseSSR`, `_RealRefraction`), foam (`_FoamEnabled`), surf (`_SurfActive`, `_ShoreDepthValid`, `_SurfCrestFoamLutActive`), SSS/glow (`_SssEnabled`, `_OceanFftActive`), ocean/FFT (`_OceanFftActive`, `_OceanWorldWaves`, `_LargeBody`), clipmap/patch/chunk (`_IsClipmap`, `_IsPatch`, `_ChunkSphereClip`, `_ChunkUseMesh`, `_ChunkFogClamp`), exclusion (`InsideExclusion`), detail normals, horizon, and debug. (By contrast, the standalone `AnalyticPool.shader` *does* use real `shader_feature_local` toggles and URP shadow multi_compiles.)

Material inputs. Visible per-material: `_Underwater`, `_Cull`, the detail-normal set, the foam set (`_FoamTex` single-tile-or-flipbook, `_FoamTexFrames/FPS`, `_FoamNormalStrength`), and `_OceanWhitecapTex`. Reflection/refraction knobs are `[HideInInspector]` (driven by the component): `_ReflectionStrength`, `_EnvReflectionIntensity`, `_UsePlanar/_UseSSR/_UseUrpProbe`, `_SSRStrength/StepSize/MaxSteps/Thickness`, `_RealRefraction`, `_RefractionDistortion`, Fresnel (`_FresnelFloor`, `_FresnelPower`), the roughness ramp, and sun sheen. Key published textures: `_WaterTex`, `_BedTex`, `_Tiles`, `_Sky`, `_CausticTex`, `_FoamMask`, `_OceanWhitecapTex`, `_PlanarReflectionTex`.

Analytic pool vs screen-space refraction. Two mutually-exclusive refraction paths per body. **Analytic pool** (`_RealRefraction = 0`, bounded body): refract into the unit pool box and sample the baked `_Tiles` floor/walls with AO, refracted-sun diffuse, projected caustics, and rim shadow, then fog by the world chord. **`AnalyticPool.shader`** is the standalone opaque pool-geometry material (Geometry queue, full URP includes, real shadow keywords, plus DepthOnly/DepthNormals prepasses) drawing the actual tile walls/floor. **Real screen-space refraction** (`_RealRefraction = 1`) instead samples the live `_CameraOpaqueTexture` (distorted by the surface normal) and fogs by scene-vs-surface eye-depth. **`ObstacleDepth.shader`** is unrelated to the surface look: it is a top-down orthographic footprint pass that encodes, per column, an obstacle's submerged thickness below the local waterline — feeding the interactive ripple sim.

Gotchas (compile constraints). The pass is UnityCG/CGPROGRAM, not full SRP, so it hand-rolls the shadow tap and `.Load`s chunk depth RTs. The include order is a dependency chain (Screen → Shadow → Specular → PoolTrace → FoamSampling → DetailNormal), and `WaterSurfaceFragStages.hlsl` must be last. For WGSL derivative uniformity, stages may only be called from `frag`'s uniform control flow; screen derivatives are hoisted *before* per-fragment branches and passed into non-uniform samplers as explicit gradients. RGBA32Float sim/foam/shadow textures are filtered manually (WebGPU point-samples them), and the shadow map and scene depth are read via explicit non-comparison point samplers to avoid an invalid bind group

(a black screen in builds). Numerous `saturate`/epsilon NaN guards and bounded loops exist specifically to keep the compiler and the WebGPU backend happy.

Shorelines, Exclusion Volumes & Water Chunks

Overview. Three capabilities all key off one idea — *how deep is the water column here?* The still-water surface minus the seabed (or shape floor) decides deep-water colour, where the shoreline sits, how waves shoal and break, whether a fragment is dry land, and where a finite chunk ends. Two bakers produce that signal on the CPU (no GPU read-back): `WaterBedBaker` (pool-frame, bounded bodies) and `WaterShoreDepthField` (world-frame, works on ocean/windowed bodies too). Both are lazily gated behind `useBedDepth` because each bake costs `resolution²` main-thread `Terrain.SampleHeight` calls.

Bed depth & shoreline. `WaterBedBaker` samples the terrain heightmap into a **pool-space** `RFloat` texture (`_BedTex`) — the seabed's pool-space Y under the body's own frame, so rotation and non-uniform extent are handled. It bakes once per enable; `Rebake()` re-runs after terrain/placement changes. Shaders read `_BedTex` for the real water-column depth (`surface − bed`), which drives deep-water colour, the shoreline gradient, and the dry-beach clip (a negative column = land, so the surface is discarded / waves clamp to zero). `WaterShoreDepthField` is the richer world-frame twin, baking `_ShoreDepthTex` (`RHalf`, still-water column depth in metres) and `_ShoreSDFTex` (`RGBAHalf`, a CPU jump-flood signed-distance field: RG = toward-shore direction, B = signed distance to shore, A = local beach slope). `EnsureBakedAndPublish()` bakes once then republishes globals every frame — even unbaked it binds `Texture2D.blackTexture` with `valid = 0`, because WebGPU never tolerates an unbound sampler. CPU arrays are kept so `LargeWaveField` (buoyancy) can `TrySampleShore` the same field the shaders see. Debug toggles: `ToggleDepthDebug()` / `ToggleSdfDebug()`. Consumption lives in `WaterShore.hlsl` / `WaterShoreMath.hlsl` (split so compute kernels, which can't include `sampler2D`, share the pure math): `ShoreSample()` returns `ShoreData`, off-field/unbaked returns a deep sentinel so consumer math collapses to open-water with no branches. `ShoalWeight`, `ShoreGreenGain` (amplitude ∝ depth^−¼), and `ShoreWarpExtra` (near-shore phase compression) all multiply by an `influence` weight; a per-body gate stops an overlapping body catching another's shoal/surf.

Exclusion volumes (Crest-style carving). `WaterExclusionVolume` (public `MonoBehaviour`, `[ExecuteAlways]`) marks an oriented box where the surface must **not** render — a hull interior, a flooded room. Volumes self-register into a static `Active` list; `WaterUniformPublisher` writes up to `MaxVolumes = 4` into globals, and over the limit the nearest to camera win (`SelectNearest`, warned once). Per-volume knobs: `edgeColor`, `edgeIntensity`, `edgeSpread`, `wallScatterBoost`, `drawWaterWalls`. `WaterExclusion.hlsl` is the single shared point test: `InsideExclusion` (surface discards inside the box), `ExclusionRayLength` (dry span the fog/god-ray integrals subtract), and refraction-aware sun shadowing (`ExclusionSunVisibility` / `ExclusionSpanSunVisibility`, the latter a closed-form convex-prism column that avoids banding). `WaterExclusionWall.shader` draws the carve boundary as **standing water** — a shared unit cube shaded with the fog's lit inscatter, `Cull Off`, `ZWrite Off`, premultiplied blend, and deliberately **no depth passes** so it stays out of `_CameraDepthTexture`; it clips to the wavy waterline (`SurfaceHeightAtXZ`). Exclusion volumes are purely visual — buoyancy, physics, and the ripple sim are untouched (a hull still floats and carves a wake). Author dry rooms disjoint; overlapping volumes double-count shared spans.

Water chunks (finite water in dry space). The inverse of a carve: `WaterVolume.Chunk.cs` turns a body into a self-contained floating body of water. `ChunkFootprint { None, Box, Sphere, Mesh }`; `IsChunk` when not `None`. Serialized knobs: `chunkDensityBoost`, `chunkRefraction`, `chunkReflectivity`, `chunkMeniscus`, `chunkFillLevel`, `chunkMesh`. The body renders its real disc surface; this partial adds a submerged **fog shell** renderer (`WaterChunkWall.shader`, a pool-space box mesh) fed *this* body's per-body block so its waterline is the exact same `SurfaceHeightAtXZ` as the disc — no seam. `SetChunkSurfaceProps` writes shape/clip/clamp onto the disc block (`_ChunkSphereClip`, `_ChunkFogClamp`, `_ChunkShape`, `_ChunkUseMesh`, `_ChunkSurfacePoolY = chunkFillLevel·2−1`). `WaterChunkPrimitive.hlsl` resolves Box/Sphere analytically in pool space (`ChunkIntersect`, `ChunkSurfaceNormalPool`). `WaterChunkWall.shader` (`Cull Front`, `ZTest Always`, no depth passes) integrates the submerged column via a coarse march + per-crossing bisection, tier-gated on `_RealRefraction` (Low = cheap veil, High/Med = refracted backdrop), with an optional `chunkMeniscus` surface-tension line. **Mesh** footprint replaces the analytic interval with a depth-RT prepass: `WaterChunkDepth.shader` writes front-face (entry) and back-face (exit) depth into two RTs the wall reads by texel `LOAD` (zero sampler cost, and it no longer depends on draw order). `WaterChunkFillAnimator` ping-pongs `chunkFillLevel` for showcase scenes; the pipeline re-derives `_ChunkSurfacePoolY` each frame.

Debug tooling. `WaterVolume.ObstacleDebug.cs` (editor-only partial) adds a "Dump Obstacle Footprint (PNG)" context menu that writes normalised, raw, and signed-delta coverage maps plus spatial stats to a `WaterDebug` folder, cross-checking via a second independent read-back — specifically to prove a uniform footprint is real, not a read-back artifact. Deleting the file removes the instrumentation.

Gotchas. Both bakes are off by default behind `useBedDepth`; a missing `Terrain` warns once. Exclusion walls and chunk shells both stay out of `_CameraDepthTexture` on purpose (adding depth passes would clamp the fog at the boundary). Chunk fog is driven per-body (a forced GPU flag) while the C# `WaterFog` flag stays false — turning on `WaterFog` would fog the wrong volume. Inter-chunk transparent sort order between two nearby analytic chunks is a render-queue limitation the Mesh depth-RT path sidesteps. The GPU/CPU constant pairs (`MaxVolumes ↔ EXCLUSION_MAX_VOLUMES`, `ChunkFootprint ↔ CHUNK_SHAPE_*`, all shore constants mirrored in `LargeWaveField`) are guarded by `WaterWaveConstantsValidator` — retune in both places, never inline.

Authoring (Water Wizard), Inspector, Validation, Cameras & Showcase

Overview. This module is the human-facing surface of the package: a one-window authoring wizard, a tabbed custom inspector for `WaterVolume`, an editor-load consistency guard for wave constants, two `[InitializeOnLoad]` editor helpers, a feature-showcase cycler, and three camera controllers. All editor entry points live under **Window ▸ AbstractOcclusion ▸ WebGpuWater** (Asset Store rules forbid a custom top-level menu).

Water Wizard (`WaterWizardWindow`). Opened via **Window ▸ AbstractOcclusion ▸ WebGpuWater ▸ Water Wizard**; its fields are `[SerializeField]` so a half-configured build survives domain reloads. `OnGUI` draws five foldout sections: **Create Water**, **Floating Objects**, **Boat**, **Splash & Crown**, and **Utilities**.

Create Water picks a base `WaterKind` — `LegacyAnalyticPool` (walls/floor/caustics, no fog), `SurfaceOnly`, `SurfaceWithFog`, or experimental `OpenWaterOcean` (large-body + horizon clipmap). It exposes a size (half-extent `Vector3`), a pool-texture picker (analytic-pool only), ocean options,

then shared options applied to any type: ripple quality, camera (`Orbit` / `Fly`), reflection with a `SkyOnly` / `SSR` / `Planar` sub-choice, foam with a gated edge-foam sub-toggle, splash, god rays, and a floor collider. `CreateWater()` guards against a duplicate root, then in a single collapsed undo group builds the root, a `BuildContext` via `WaterBuildKit.CreateContext`, and a primary `WaterVolume` body; it then applies ripple quality, base type, custom pool tile, reflection flags, foam, the open-water/body-type archetype (large horizontal extents auto-arm open water past `largeBodyThreshold`), and the camera mode.

Floating Objects retrofits existing objects as `Floatable` (Rigidbody + `WaterInteractable` + `WaterBuoyancy` + optional `WaterSplash` + `WaterMembership`, tuned by a Light/Normal/Heavy preset and an Advanced foldout) or `InteractableStatic`; a sub-panel spawns a fresh Cube/Sphere/Capsule/CustomMesh prop, fully wired. *Boat* creates a drivable hull with probe buoyancy + `BoatController`, an optional yaw-only chase camera, and a "Dry Interior" `WaterExclusionVolume`. *Splash & Crown* adds the shared emitter and `WaterSplash` triggers. *Utilities* delegates to `WaterSceneBuilder`: Create WaterVolume Prefab, Add Foam Particles To Selected, Assign Foam Textures To Scene Water, Upgrade Splash Materials (lit), and Add Water Body (secondary).

Builder internals. `WaterBuildKit` is the shared editor-only generator. `CreateContext` = `TryBuildSharedAssets` (create-once meshes/sky/tiles/`WaterQuality`/materials, written into the open project's `Assets/WebGLWater/Generated`, never the read-only package) + `RigScene` (reuses `Camera.main`, adds `OrbitCamera`, a Sun, and a `WaterSplashEmitter` with a crown). `CreateWaterBody` assembles a "Frame" GameObject carrying the `WaterVolume` (its transform *is* the volume frame) plus a "Renderers" group (above/under surfaces, optional analytic pool, god-ray box). `WireWaterVolumeAssets` is the single source for a body's asset slots; `WireWaterVolumeFrom` copies from a live body for secondary clones. Shaders load through `TryLoadShaders` (fails fast with a dialog if the surface/caustics/compute assets are missing). `WaterSceneBuilder` holds the retrofit operations, including `AddSecondaryBody` (a non-primary body offset from the primary, cloning its renderers via `MaterialPropertyBlock`).

Custom inspector. The `[CustomEditor(typeof(WaterVolume))]` is a partial split across files. `WaterVolumeEditor.cs` draws the scene-view gizmo (oriented wire box, surface rectangle, sim-window preview) and face-slider handles for extent. `WaterVolumeEditor.Inspector.cs` orchestrates a cyan header, a Pond/Lake/Ocean selector with "Apply defaults", the Jerlov colour selector, then a five-tab bar — **Core**, **Look**, **Waves & Wind**, **Shore & Surf**, **Performance** — with foldout state persisted via `SessionState` and sections greyed by body type (advisory only). `WaterVolumePropertyPaths` is a single registry of the raw serialized paths shared by wizard and inspector, so a field rename breaks loudly in one place. `WaterEditorUI` supplies the palette, sections, tab bar, and footer. `WaterVolumeEditor.Chunk.cs` is the only inspector for the hidden chunk fields; `.Jerlov.cs` writes physical absorption/albedo into fog + scatter.

Wave-constant validator. `WaterWaveConstantsValidator` is an `[InitializeOnLoad]` read-only watcher that on editor load parses paired constants and reports drift across five tables: `LBW_*`, `SURF_*`, `SHORE_*` (all ↔ `LargeWaveField`), `BURST_*` (`WaterFoamParticles.compute` ↔ `WaterSplashEmitter`), and `EXCLUSION_MAX_VOLUMES` (↔ `WaterExclusionVolume.MaxVolumes`). Values must match within `1e-5` relative tolerance; any mismatch, missing, or renamed constant is a `Debug.LogError`. It changes no runtime behaviour and no files — it just replaces "remember to edit both files" with a hard check.

Editor support. `WaterChunkShaderRegistration` (`[InitializeOnLoad]`) idempotently adds `WaterChunkWall` and `WaterChunkDepth` to the Always Included Shaders so code-only

(`Shader.Find`) chunk shaders survive player builds. `WaterEditorPreviewDriver` (`[InitializeOnLoad]`) pumps the player loop + `SceneView.RepaintAll` at ~60 fps while a water body is alive, so ripples/waves/caustics/foam run live in edit mode; it is toggleable via a **Live Water Preview** menu item.

Showcase system. `WaterShowcaseController` cycles inactive `WaterShowcaseStation` templates — exactly one is instantiated as a live clone at a time, so each visit starts fresh and teardown is one destroy. `N` / `M` or arrow keys cycle; the editor adds Previous/Next buttons. Each `WaterShowcaseStation` holds a display name, description, orbit framing, and an optional sun override; an on-screen caption shows the station index/name/description. `WaterShowcaseMover` drives a kinematic circular path for props; `WaterShowcaseDripper` (`[RequireComponent(typeof(WaterRippleEmitter))]`) calls `Emit()` at a fixed interval so a pond ripples on its own.

Cameras. `OrbitCamera` orbits a pivot (drag to rotate, scroll/pinch to zoom, clamped pitch/distance). `FlyCamera` is free-fly (WASD on the view plane, Q/E down/up, hold right-mouse to look, Shift to boost; on touch, left-half drag = move stick, right-half = look, two-finger pinch = altitude). `SimpleFollowCamera` is the boat chase camera: follows a target from a fixed local offset around yaw only (no roll/pitch nausea), frame-rate-independent, aiming slightly above the target.

Mobile, WebGPU & Performance

The runtime auto-selects the **Low** tier on WebGL/WebGPU/mobile (via `WaterQuality.Probe`) and disables a body cleanly if the device lacks compute shaders or float render textures. Because the simulation is compute-based, WebGL builds require **WebGPU**; browsers or devices without it get a clear message instead of a crash. The sim is frame-rate independent — wave speed is identical in a 30 fps build and a 144 fps editor session.

Because each tier changes resolutions and scales (sim/caustic resolution, render scale, god-ray steps, wave count, refraction, foam-particle caps), **the High and Low tiers usually need different visual-tuning values to look correct**. A look dialed in at High can read too strong, too weak, or too coarse at Low. Treat per-tier tuning as expected. **To preview what will actually render on mobile, set the Quality asset to Force Low** — that is the only way to see the resolution, render scale, and particle caps a device build will use.

Where `AsyncGPUReadback` is unavailable, buoyancy falls back to the analytic waterline: objects still float and ride the waves, they just don't react to interactive ripples or object displacement. Planar reflection is a second camera render *per body*, so it does not scale to many bodies — use SSR + a reflection probe for multi-body scenes and reserve planar for a single hero body (the `MaxActivePlanarBodies = 3` cap enforces this automatically).

Verified hardware (Low tier, with foam, caustics, and god rays enabled): ~30 fps on a Honor X6 and a Redmi Pad SE (Snapdragon 4G-class, Adreno 610), 30+ fps on a Samsung Galaxy A17.

Demo Scenes

The demos ship as a Package Manager sample (`Samples~/Demos`); import **Demo Scenes** from the package's Samples tab. The scene set (numbers are the in-project ordering; several are marked EXPERIMENTAL in the file names):

`0. Original Pool` · `1. Classic Pool` · `2. Deep Lake` · `3. Terrain Lake` *(experimental)* · `4. Multi-Lake` · `5. Underwater` · `6. Open Water` *(experimental)* · `7. Reflections Trio` · `8. Buoyancy Object Pool` · `9. MultiLevel Multipool` · `10. WebGPU Pool` · `11. Splashes and Foam` · `12. Ocean Demo` · `13. Buoyancy StressTest` · `14. Island Demo` · `15. Boat Demo` · `16. Chunk Demo` · `17. Exclusion Demo` · `18. Showcase`.

Together they exercise every module: the analytic pool and caustics (Classic/Original Pool), depth extinction (Deep Lake), multi-body per-body property blocks (Multi-Lake, MultiLevel), the underwater fog/caustic/god-ray stack (Underwater), the FFT ocean and horizon clipmap (Ocean Demo, Open Water), buoyancy and two-way coupling at scale (Object Pool, Stress Test), the boat controller with wake and dry-interior exclusion (Boat Demo), finite water chunks (Chunk Demo), carving (Exclusion Demo), and the station cycler (Showcase). Each demo's generated meshes, sky, and materials import alongside it into `Assets/Samples/…`.

Cross-Cutting Engineering Notes

A few principles recur across modules and are worth collecting in one place.

WebGPU is the design floor, not an afterthought. Read-write compute targets are single-channel `r32`; `RGBA32Float` textures are filtered by hand in shaders; every sampler is bound to a black fallback rather than left null; shadow maps and scene depth are read with explicit point samplers; group-shared reductions replace mip chains that would silently point-sample; and shader stages hoist screen-space derivatives out of non-uniform control flow to satisfy WGSL uniformity. Several of these fixes trace to concrete build bugs (a black screen, a popping plane, vanishing shadow shafts) recorded in the code comments.

One derivation, two sinks. Per-body state is derived once in `WaterUniformPublisher` and written either to a `MaterialPropertyBlock` (that body) or to a global mirror (the primary), so the local and global copies can never drift. `_WaveTime` and `_Sky` are always per-body to avoid last-writer-wins across bodies with different clocks and skies.

CPU/GPU parity is enforced, not trusted. Wherever the surface height is evaluated on both the CPU (buoyancy) and the GPU (rendering), the math and constants are duplicated byte-for-byte and cross-checked at editor load by `WaterWaveConstantsValidator`. The single shared bilinear filter (`WaterFieldSampling`) is reused by every CPU mirror so sampling can't diverge either.

Fast Enter Play Mode is fully supported. All scene-lifetime static state (`Bodies`, `Primary`, scheduler caches) resets via `[RuntimeInitializeOnLoadMethod]` / `SubsystemRegistration` before each play session, so disabling domain reload never leaves stale references.

The package stays read-only. Compute shaders load from the package; everything authored (meshes, materials, sky, quality asset) is written into the consuming project's `Assets/`, editable and owned by the user.

Troubleshooting

Symptom	Fix
God-ray shafts invisible	Enable **Transparent Receive Shadows** on the active URP asset.
No refraction / SSR	Enable **Depth Texture** and **Opaque Texture** on the URP asset.
Water looks blocky in a build	The device lacks float32 filtering; the package handles this automatically — ensure you are on 1.0.0+.
Nothing floats	The object needs a `Rigidbody`, a `Collider`, and `WaterBuoyancy`; the scene needs a `WaterVolume` whose footprint contains it.
A boat is pushed by its own wake	Set `ignoreInteractiveRipples` on its `WaterBuoyancy` (it then samples the analytic surface).
Ripples too fast / slow	Tune `waveSpeed` / `stepsPerFrame` on the `WaterVolume` (60 fps reference — identical in builds).
High-tuned look wrong on mobile	Tune per tier; set the Quality asset to **Force Low** to preview the device look.
Planar reflection too costly with many bodies	Only the nearest `MaxActivePlanarBodies = 3` render planar; use SSR + a reflection probe for the rest.
Foam looks different at Low tier	Foam-particle count is capped per tier (`MaxFoamParticles`); re-tune density for Low.
A carved room shows unfogged scene	Keep exclusion volumes disjoint and set `drawWaterWalls` off for volumes with real covering geometry.

Credits & License

Original concept, design, and GLSL shaders: © 2011 **Evan Wallace** — https://madebyevan.com/webgl-water/ — released under the **MIT License**. The GPU height-field simulation, the in-shader ray-traced reflection/refraction, and the projected caustics are his ideas.

Unity 6 / URP adaptation and enhancements: © 2026 **Abstract Occlusion**, also released under the **MIT License**. This adaptation is provided in the same spirit as the original; if you use it, please keep the credit to Evan Wallace for the original work.

Package: `com.abstractocclusion.webgpuwater` · Support: abstractocclusion@outlook.com · https://abstractocclusion.com